

Week05 – Math and Logic Instruction, MQTT, BLE Remote
PC ____ รหัส ____ ชื่อ-สกุล ____
เปลี่ยนชื่อไฟล์เป็น “B6501234-นายไมโคร รายงานแลป-Lab05.pdf” ส่งงาน Report ที่ >> https://forms.gle/8fnzBjS5vdmg9bq26

M21. Test and Discuss Results >> ก่อนโปรแกรมทำงานให้มีค่าเริ่มต้นตามหัวตาราง

Assembly instruction		PC	DPTR	Acc	PSW	C	AC	21H	24H	8888H
Item	Initial	0000	1234H	56H	00H	0	0	78H	9AH	BCH
1	ORG 0000H	0000H	1234H	56H	00H	0	0	78H	9AH	BCH
2	MOV A, #0FEH									
3	ANL A, #0F0H									
4	ORL A, #005H									
5	RR A									
6	INC A									
7	RL A									
8	SETB Acc.4									
9	DEC A									
10	SWAP A									
11	CRL C									
12	ADD A, #0FFH									
13	RLC A									
14	SETB 21H									
15	ANL A, 24H									
16	MOV DPTR, #8888H									
17	MOVX @DPTR, A									
18	SJMP \$									
19	END									

1.Math Instruction	2.Logic Instruction	3. HEX2BCD BCD2HEX	4. MQTT	5. BLE Remote

Mission-1/5. Math Instruction (TS Emulator)

- 1.1 จากโปรแกรม “Lab41.Asm” ให้นักศึกษาปรับปรุงโปรแกรมเพื่อรวมค่าต่างๆแล้ว
- ☐ รวมค่าใน R0-R7 แบบไบนารี เก็บผลลัพธ์ที่ได้ในหน่วยความจำภายนอกที่ 9500H-9501H
 - ☐ รวมค่าในหน่วยความจำภายในตำแหน่งที่ 40H-47H แบบไบนารี
 - ☐ รวมค่าในหน่วยความจำภายนอกตำแหน่งที่ 9000H-9007H แบบไบนารี
- 1.3 ☐ ให้นักศึกษาเขียนโปรแกรมบวกเลขฐานสอง จำนวน 4 ไบต์ แบบไบนารี โดยมีข้อกำหนด ดังนี้
- ตัวตั้ง = หน่วยความจำภายนอกที่ตำแหน่ง 9001H (MSB) - 9004H (LSB)
 ตัวบวก = หน่วยความจำภายนอกที่ตำแหน่ง 9011H (MSB) - 9014H (LSB)
 ผลลัพธ์ = หน่วยความจำภายนอกที่ตำแหน่ง 9020H (MSB) - 9024H (LSB)
- หมายเหตุ: การบวกต้องคำนึงถึงตัวทศระหว่างไบต์ และต้องมีการบันทึกผลลัพธ์ไบต์ที่ 5 ด้วย
- 1.2 ☐ ให้นักศึกษาเขียนโปรแกรมเพื่อดำเนินการลบเลขฐานสิบหก จำนวน 3 ตัว ดังต่อไปนี้
- ตัวตั้งกำหนดไว้ก่อนรันโปรแกรมที่หน่วยความจำภายนอกตำแหน่ง 8800H
 ตัวลบกำหนดไว้ก่อนรันโปรแกรมที่หน่วยความจำภายในตำแหน่ง 31H
 คัดลอกข้อมูลจากตำแหน่ง 8800H มาเก็บไว้ในหน่วยความจำภายในตำแหน่ง 21H
 ดำเนินการลบ แล้วเก็บผลลัพธ์ที่ได้ไว้ในหน่วยความจำตำแหน่ง 40H - 43H
- 1.5 ☐ เขียนโปรแกรมเพื่อทำการคูณเลข 2 จำนวน ตามข้อกำหนด ดังนี้
- ตัวตั้งเป็นเลข 2 ไบต์ที่เก็บใน R4:R3 (R4 =MSB, R3=LSB)
 ตัวคูณเป็นเลข 1 ไบต์ที่เก็บใน R6
 ผลลัพธ์ที่ได้ไปเก็บในหน่วยความจำภายนอก โดยให้ค่าไบต์ต่ำเริ่มที่ตำแหน่ง 9000H และให้โปรแกรมเริ่มต้นที่แอดเดรส 8200H

Picture(รูปถ่ายการทำงาน)
Capture Code
Text Code

Mission-2/5. Logic Instruction (TS Emulator)

☒ เขียนโปรแกรมตามโปรแกรม “Lab42.Asm” คอมไพล์และโหลดโปรแกรมด้วย TS-Emulator ทดสอบการทำงานของโปรแกรมแบบ Single Step สังเกตการเปลี่ยนแปลงข้อมูลในหน่วยความจำที่เกี่ยวข้องและบันทึกผลการรันโปรแกรมลงในตารางบันทึกผล

2.2 ☐ กำหนดให้ Register B มีค่าอยู่ก่อนแล้ว ให้นักศึกษาทำการนับจำนวนเลข 1 ในค่าใน Register B เก็บผลลัพธ์ที่ได้ใน R7

เช่น B=F7H → 1111 0111

Run แล้วได้ R7=01H

2.3 ☐ กำหนดให้ Register DPTR มีค่าอยู่ก่อนแล้ว ให้นักศึกษาทำการนับจำนวนเลข 0 ในค่าใน Register DPTR เก็บผลการนับแบบ BCD ที่รีจิสเตอร์ B

เช่น DPTR=1234H → 0001 0010 0011 0100 Run แล้วได้ B=11H (11 ตัว)

Mission-3/5. HEX ↔ BCD (TS Emulator)

3.1 ☐ เขียนนักศึกษาโปรแกรมเพื่อรวมเลขแบบ BCD ระหว่าง DPTR R0 และ R1 รวมเสร็จแล้วให้เก็บค่าผลลัพธ์ที่หน่วยความจำภายในตำแหน่งใดๆ ก็ได้ {ระหว่าง 30H-7FH}

เช่น DPTR=1234H, R0=00H และ R1=11H
Result = 1234+00+11 = 1245H เป็นต้น

3.2 ☐ ใส่ค่าเริ่มต้นลงในรีจิสเตอร์ DPTR เพื่อเป็นค่าที่ต้องการแปลงจากไบนารี (HEX) ไปเป็นเลขบีซีดี (BCD)

3.3 ☐ ใส่ค่าเริ่มต้นลงในรีจิสเตอร์ DPTR เพื่อเป็นค่าที่ต้องการแปลงจากเลขบีซีดี (BCD) ไปเป็นเลขไบนารี (HEX)

Picture(รูปถ่ายการทำงาน)
Capture Code
Text Code

Mission-4/5. MQTT

- 4.1 กำหนด Topic Name เป็นรหัสและชื่อของนักศึกษาเอง เช่น Topic = B3601234_Wichai
- 4.2 อ่านค่า DHT-22 แล้วส่งไปยัง MQTT Broker ทุกๆ 2 วินาที
- 4.3 เพิ่มเติมให้ทำงานรับคำสั่งเพื่อควบคุม LED จำนวน 4 ตัว
- 4.4 เพิ่มเติมให้รับคำสั่งวิดิซ์ กำหนด
 - หากกด SW1 ให้แจ้ง Overheat Alarm
 - หากกด SW2 ให้แจ้ง Intruders Alarm

Picture(รูปถ่ายการทำงาน) จำนวนอย่างน้อย 2 รูป
Capture Code
Text Code

Mission-5/5. BLE Remote

- 5.1 ติดตั้ง Dabble บน Smart Phone: <https://thetempedia.com/product/dabble/>
- 5.2 ใช้ ESP32 BLE ในการสื่อสารกับ Mobile Application เพื่อควบคุมการปิดเปิด LED จำนวน 8 ตัว

Picture(รูปถ่ายการทำงาน) จำนวนอย่างน้อย 2 รูป
Capture Code
Text Code

1/7. MCS-51 Assembly Code:

; ==== Lab41.Asm ====

```

HResult EQU    20H
LResult EQU    21H

    ORG    0000H
    MOV    HResult,#00    ; Clear Result
    MOV    LResult,#00

    CLR    C
    MOV    A,R0
    ADDC   A,LResult
    MOV    LResult,A
    MOV    A,#00
    ADDC   A,HResult
    MOV    HResult,A

    CLR    C
    MOV    A,R1
    ADDC   A,LResult
    MOV    LResult,A
    MOV    A,#00
    ADDC   A,HResult
    MOV    HResult,A

    CLR    C
    MOV    A,R2
    ADDC   A,LResult
    MOV    LResult,A
    MOV    A,#00
    ADDC   A,HResult
    MOV    HResult,A

    JMP    $
    END

```

; ==== Lab42.Asm ====

```

    ORG    0000H    ; 1
    MOV    A,#0FEH  ; 2
    ANL    A,#0F0H  ; 3
    ORL    A,#005H  ; 4

    RR     A         ; 5
    INC    A         ; 6
    RL     A         ; 7
    SETB   Acc.4     ; 8
    DEC    A         ; 9
    SWAP   A         ; 10
    CLR    C         ; 11
    ADD    A,#0FFH   ; 12
    RLC    A         ; 13

    SETB   21H       ; 14
    ANL    A,24H     ; 15
    MOV    DPTR,#8888H ; 16
    MOVX   @DPTR,A   ; 17

    SJMP   $         ; 18
    END           ; 19

```

2/7. MCS-51 เพื่อหา ทฤษฎีที่เกี่ยวข้อง

1. การใช้คำสั่งการบวก การลบ การคูณ และการหาร

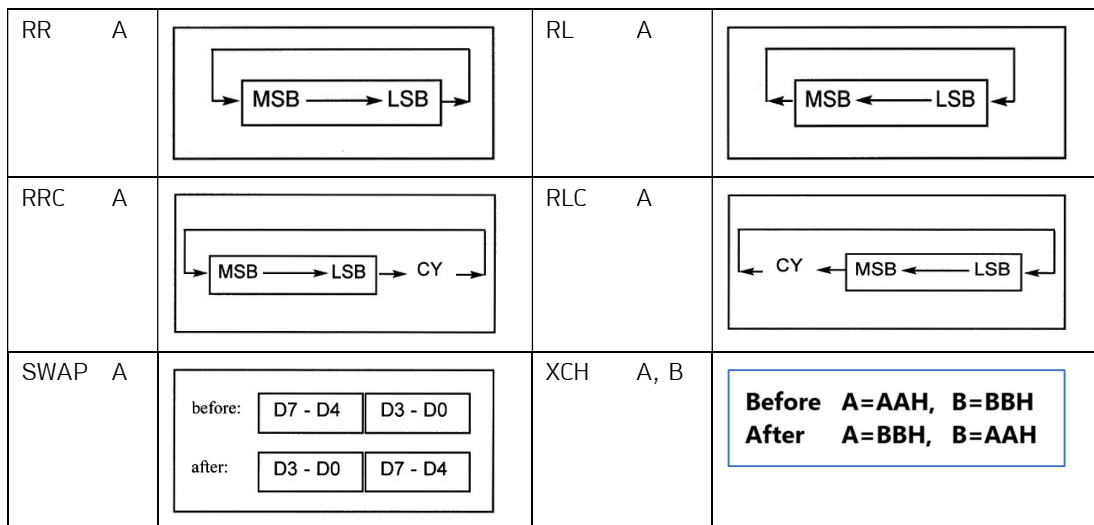
ADD **A, Source**
ADDC **A, Source**
DA **A**
SUBB **A, Source**
MUL **AB ; Result BA**
DIV **AB ; Result A, Remain B**

2. การใช้คำสั่งการคำนวณเชิงตรรกะ

AND **A, Source**
ORL **A, Source**
XRL **A, Source**
CPL **A**

3. การใช้คำสั่งเกี่ยวกับการหมุนบิต

RR **A ;rotate right A**
RL **A ;rotate left A**
RRC **A ;rotate right with carry**
RLC **A ;rotate left with carry**
SWAP **A ; Swap**
XCH **A,B ; Exchange**



4. การใช้คำสั่งเพื่อศึกษาการแปลงระหว่างเลขฐานสิบหกกับเลขฐานสิบ

HEX2BCD → FDH (253D)

Step1: FDH/0AH → Result =19H Remain =03H
Step2: 19H/0AH → Result =02H Remain =05H
Step3: 02H/0AH → Result =00H Remain =02H

BCD2HEX → 45H (2DH)

High Nipple * 10 + Low Nipple = Result
04H * 0AH + 05H = 2DH

3/7. คำถามในการทดลอง

ตอนที่ 1 คำสั่งการบวก ลบ คูณ และหาร

1.1 โปรแกรม “Lab41.Asm” เป็นโปรแกรมสำหรับรวมค่า R0, R1 และ R2 ทดสอบการทำงานด้วย TS-Emulator

- ก่อนรันโปรแกรมกำหนดค่า $R0=R1=R2=0FFH$
- ผลลัพธ์เก็บที่แอดเดรส 20H(MSB)=02H ,21H(LSB)=FDH { $FF+FF+FF=02FD$ }

1.2 จากโปรแกรม “Lab41.Asm” ให้นักศึกษาปรับปรุงโปรแกรมเพื่อรวมค่าต่างๆแล้วเก็บผลลัพธ์ที่ได้ในหน่วยความจำภายนอกที่ 9500H-9501H

1.2.1 รวมค่าใน R0-R7 แบบไบนารี

1.2.2 รวมค่าในหน่วยความจำภายในตำแหน่งที่ 40H-47H แบบไบนารี

1.2.3 รวมค่าในหน่วยความจำภายนอกตำแหน่งที่ 9000H-9007H แบบไบนารี

1.3 ให้นักศึกษาเขียนโปรแกรมเพื่อดำเนินการลบเลขฐานสิบหก จำนวน 2 ตัว ดังต่อไปนี้

ตัวตั้งกำหนดไว้ก่อนรันโปรแกรมที่หน่วยความจำภายนอกตำแหน่ง 8800H

ตัวลบกำหนดไว้ก่อนรันโปรแกรมที่หน่วยความจำภายในตำแหน่ง 31H

คัดลอกข้อมูลจากตำแหน่ง 8800H มาเก็บไว้ที่หน่วยความจำภายในตำแหน่ง 21H

ดำเนินการลบ แล้วเก็บผลลัพธ์ที่ได้ไว้ในหน่วยความจำตำแหน่ง 40H และ 41H

1.4 ให้นักศึกษาเขียนโปรแกรมบวกเลขฐานสอง จำนวน 4 ไบต์ แบบไบนารี โดยมีข้อกำหนด ดังนี้

ตัวตั้ง = หน่วยความจำภายนอกที่ตำแหน่ง 9001H (MSB) - 9004H (LSB)

ตัวบวก = หน่วยความจำภายนอกที่ตำแหน่ง 9011H (MSB) - 9014H (LSB)

ผลลัพธ์ = หน่วยความจำภายนอกที่ตำแหน่ง 9020H (MSB) - 9024H (LSB)

หมายเหตุ: การบวกต้องคำนึงถึงตัวทดระหว่างไบต์ และต้องมีการบันทึกผลลัพธ์ไบต์ที่ 5 ด้วย

1.5 เขียนโปรแกรมเพื่อทำการคูณเลข 2 จำนวน ตามข้อกำหนด ดังนี้

ตัวตั้งเป็นเลข 2 ไบต์ที่เก็บใน R4:R3 (R4 =MSB, R3=LSB)

ตัวคูณเป็นเลข 1 ไบต์ที่เก็บใน R6

ผลลัพธ์ที่ได้ไปเก็บในหน่วยความจำภายนอก โดยให้ค่าไบต์ต่ำเริ่มที่ตำแหน่ง 9000H และให้โปรแกรมเริ่มต้นที่แอดเดรส 8200H

ตอนที่ 2 คำสั่งการคำนวณเชิงตรรกะและคำสั่งเกี่ยวกับการหมุนบิต

2.1 การคำนวณเชิงตรรกะส่วนใหญ่ใช้เพื่อต้องการตรวจสอบระดับบิตหรือการจัดการระดับบิตในกรณีที่รีจิสเตอร์ไม่สามารถจัดการระดับบิตได้

- กรณีที่ต้องการให้รีจิสเตอร์ TMOD เฉพาะ 4 บิตล่างเป็น 0101B จะใช้ชุดคำสั่งดังนี้

```
ANL    TMOD,#11110000B
ORL    TMOD,#00000101B
```

- กรณีที่ต้องการตรวจสอบว่าให้รีจิสเตอร์ Acc มีค่าเท่ากับ 10110101B หรือไม่

```
XRL    A,#10110101B
```

ผลการทำงานชุดคำสั่งนี้

- ถ้าผลลัพธ์ Acc = 00000000B แสดงว่าก่อนการทำคำสั่งนี้ค่า A=10110101B
- ถ้าผลลัพธ์ Acc $\neq$ 00000000B แสดงว่าก่อนการทำคำสั่งนี้ค่า A $\neq$ 10110101B

2.2 กำหนดให้ Register B มีค่าอยู่ก่อนแล้ว ให้นักศึกษาทำการนับจำนวนเลข 1 ในค่าใน Register B เก็บผลลัพธ์ที่ได้ใน R7

เช่น B=F7H $\rightarrow$ 1111 0111

Run แล้วได้ R7=01H

- 2.3 กำหนดให้ Register DPTR มีค่าอยู่ก่อนแล้ว ให้นักศึกษาทำการนับจำนวนเลข 0 ในค่าใน Register DPTR เก็บผลการนับแบบ BCD ที่รีจิสเตอร์ B

เช่น DPTR=1234H → 0001 0010 0011 0100 Run แล้วได้ B=11H (11 ตัว)

- 2.4 ให้นักศึกษาเขียนโปรแกรมเพื่อทำการนับว่าค่าในหน่วยความจำตำแหน่ง 40H – 7FH มีจำนวนบิตที่เป็น 1 กี่บิต กำหนดหน่วยความจำสำหรับเก็บผลลัพธ์ด้วยตัวเอง
{เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}

ตอนที่ 3 การแปลงเลขฐานสิบหกเป็นฐานสิบ

3.1 เขียนนักศึกษาโปรแกรมเพื่อรวมเลขแบบ BCD ระหว่าง DPTR R0 และ R1 รวมเสร็จแล้วให้เก็บค่าผลลัพธ์ที่หน่วยความจำภายในตำแหน่งใดก็ได้ {ระหว่าง 30H-7FH }

เช่น DPTR=1234H, R0=00H และ R1=11H
Result = 1234+00+11 = 1245H เป็นต้น

3.2 ใส่ค่าเริ่มต้นลงในหน่วยความจำในข้อมูลตำแหน่ง 8500H เพื่อเป็นค่าที่ต้องการแปลงจากไบนารี(HEX) เป็นเลขปียซีดี(BCD)

3.3 ใส่ค่าเริ่มต้นลงในรีจิสเตอร์ R1 เพื่อเป็นค่าที่ต้องการแปลงจากเลขบีซีดี(BCD)เป็นเลขไบนารี(HEX) โดยค่านี้ต้องเป็นเลข BCD เท่านั้น

3.4 ให้นักศึกษาเขียนโปรแกรมแปลงเลขบีซีดีขนาด 8 บิต ที่เก็บไว้ในรีจิสเตอร์ R1 เป็นเลขไบนารี เก็บไว้ในหน่วยความจำภายในแอดเดรส 30H

4/7. คำถามท้ายการทดลอง

Q1. ให้นักศึกษาเขียนโปรแกรมเพื่อทำการนับว่าค่าในหน่วยความจำตำแหน่ง 40H – 7FH มีจำนวนไบต์ที่เป็น FF กี่ไบต์
{เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}

<pre> ORG 0000H MOV B,#00 ; Result MOV R0,#40H ; Start MOV R1,#40H ; 7F-40+1 LOOP: MOV A,@R0 XRL A,#0FFH JNZ xLoop INC B xLoop: INC R0 DJNZ R1,LOOP JMP \$ END </pre>	40H - 7FH มีจำนวน 64 ไบต์ คำตอบมากที่สุด คือ 64D, 40H → 1ไบต์
--	--

Q2. ให้นักศึกษาเขียนโปรแกรมเพื่อทำการนับว่าค่าในหน่วยความจำตำแหน่ง 40H – 7FH มีจำนวนนิบเบิล(4Bit)ที่เป็น F กี่นิบเบิล {เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}

	ORG 0000H MOV B,#00 ; Result MOV R0,#40H ; Start MOV R1,#40H ; 7F-40+1	40H - 7FH มีจำนวน 64 ไบต์ มีจำนวน 128 นิบเบิล คำตอบมากที่สุด คือ 128D, 80H → 1ไบต์
HNib:	MOV A,@R0 SWAP A ANL A,#0FH XRL A,#0FH JNZ LNib INC B	
LNib:	MOV A,@R0 CPL A ANL A,#0FH JNZ XLoop INC B	
XLoop:	INC R0 DJNZ R1,HNib JMP \$ END	

Q3. ให้นักศึกษาเขียนโปรแกรมเพื่อทำการนับว่าค่าในหน่วยความจำตำแหน่ง 40H – 7FH มีจำนวนบิตที่เป็น 1 กี่บิต
{เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่บิต}

<pre> ORG 0000H MOV DPTR,#0000 ; Result MOV R0,#40H ; Start MOV R1,#40H ; 7F-40+1 LOOP: MOV R2,#8 ; 8 Bit MOV A,@R0 _BitC: RLC A JNC _Zero INC DPTR _Zero: DJNZ R2,_BitC INC R0 DJNZ R1,LOOP JMP \$ END </pre>	40H – 7FH มีจำนวน 64 ไบต์ มีจำนวน 512 บิต คำตอบมากที่สุด คือ 512D, 0200H → 2ไบต์
--	--

Q4. ให้นักศึกษาเขียนโปรแกรมเพื่อทำการนับว่าค่าในหน่วยความจำตำแหน่ง 40H – 7FH มีจำนวนบิตที่เป็น 0 กี่บิต ให้ตอบเป็นเลข BCD {เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}

<pre> ORG 0000H MOV DPTR,#0000 ; Result MOV R0,#40H ; Start MOV R1,#40H ; 7F-40+1 LOOP: MOV R2,#8 ; 8 Bit MOV A,@R0 _BitC: RLC A JC _Zero CALL BCDInc_DPTR _Zero: DJNZ R2,_BitC INC R0 DJNZ R1,LOOP JMP \$ BCDInc_DPTR: CLR C MOV A,DPL ADDC A,#01 DA A MOV DPL,A MOV A,DPH ADDC A,#00 DA A MOV DPH,A RET END </pre>	40H - 7FH มีจำนวน 64 ไบต์ มีจำนวน 512 บิต คำตอบเป็นเลข BCD คือ 0512H → 2ไบต์
---	--

Q5. ให้นักศึกษาเขียนโปรแกรมแปลงเลขไบนารี ขนาด 8 บิต ที่เก็บไว้ในหน่วยความจำ 8500H

- เป็นเลข Unpacked BCD ขนาด 3หลัก เก็บไว้ในหน่วยความจำ R5, R6, R7
- เป็นเลข Packed BCD ขนาด 2ไบต์/3หลัก เก็บไว้ในหน่วยความจำ DPTR

<pre> ORG 0000H MOV DPTR,#8500H MOVX A,@DPTR ; 123D CALL Hex_Acc2BCD_DPTR JMP \$ </pre> Hex_Acc2BCD_DPTR: <pre> MOV B,#10 ; DIV AB ; A=12, B=03 MOV R7,B ; 03 MOV B,#10 ; DIV AB ; A=01, B=02 MOV R6,B ; 02 MOV R5,A ; 01 MOV DPH,R5 ; 01 MOV A,R6 SWAP A ORL A,R7 MOV DPL,A ; 23 RET END </pre>	EAH → 234D 7BH → 123D HEX2BCD → FDH (253D) Step1: FDH/0AH → Result =19H Remain =03H Step2: 19H/0AH → Result =02H Remain =05H Step3: 02H/0AH → Result =00H Remain =02H BCD2HEX → 45H (2DH) High Nipple * 10 + Low Nipple = Result 04H * 0AH + 05H = 2DH
--	--

Q6. กำหนดให้ป้อนค่าใน Register B ก่อนรันโปรแกรม และให้นักศึกษาเขียนโปรแกรมเพื่อบวกค่าจาก 0 ถึงค่าที่เก็บใน Register B กำหนดหน่วยความจำสำหรับเก็บผลลัพธ์ด้วยตัวเอง

{เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}

เช่น Register B = 0FH

$0 + 1 + 2 + \dots + 9 + A + B + C + D + E + F = 78H$ เป็นต้น

<pre> ORG 0000H MOV DPTR,#0000H ; Result LOOP: MOV A,B JZ END_ADD_LOOP CLR C ADDC A,DPL MOV DPL,A MOV A,DPH ADDC A,#0 MOV DPH,A DEC B JMP LOOP END_ADD_LOOP: SJMP \$ END </pre>	ค่า B ที่ทำให้คำตอบสูงสุด คือ FF(255) ผลการบวก $0 + \dots + 255 = 32640(7F80H)$ → 2ไบต์
---	---

Q7. กำหนดให้ป้อนค่าใน Register B ก่อนรันโปรแกรม และให้นักศึกษาเขียนโปรแกรมเพื่อรวมตั้งแต่ค่าที่เก็บใน Register B จนถึง 99H

- หากค่าที่ป้อน มากกว่า 99H ให้คำตอบเป็น F ทุกนิบเปิล
- กำหนดหน่วยความจำสำหรับเก็บผลลัพธ์ด้วยตัวเอง

{เมื่อโปรแกรมทำงานจะให้ผลลัพธ์สูงสุดเท่าใด, ควรเก็บผลลัพธ์กี่ไบต์}

เช่น	Register B = 9AH	→ <Over 99H>	= FFFF...
	Register B = 99H	→ 99H	= 99H
	Register B = 98H	→ 98H + 99H	= 131H

<pre> ORG 0000H MOV DPTR,#0FFFFH ; Result CLR C MOV A,#99H SUBB A,B JC END_PROCESS LOOP: MOV DPTR,#0000H ; Result CLR C MOV A,DPL ADDC A,B MOV DPL,A MOV A,DPH ADDC A,#0 MOV DPH,A MOV A,B XRL A,#99H JZ END_PROCESS INC B JMP LOOP END_PROCESS: SJMP \$ END </pre>	ค่า B ที่ทำให้คำตอบต่ำสุด คือ 99H(153) ผลการบวก 99H(153) = 153(99H) → 1ไบต์ ค่า B ที่ทำให้คำตอบสูงสุด คือ 00 ผลการบวก 0+...+153 = 11781(2E05H) → 2ไบต์
--	--

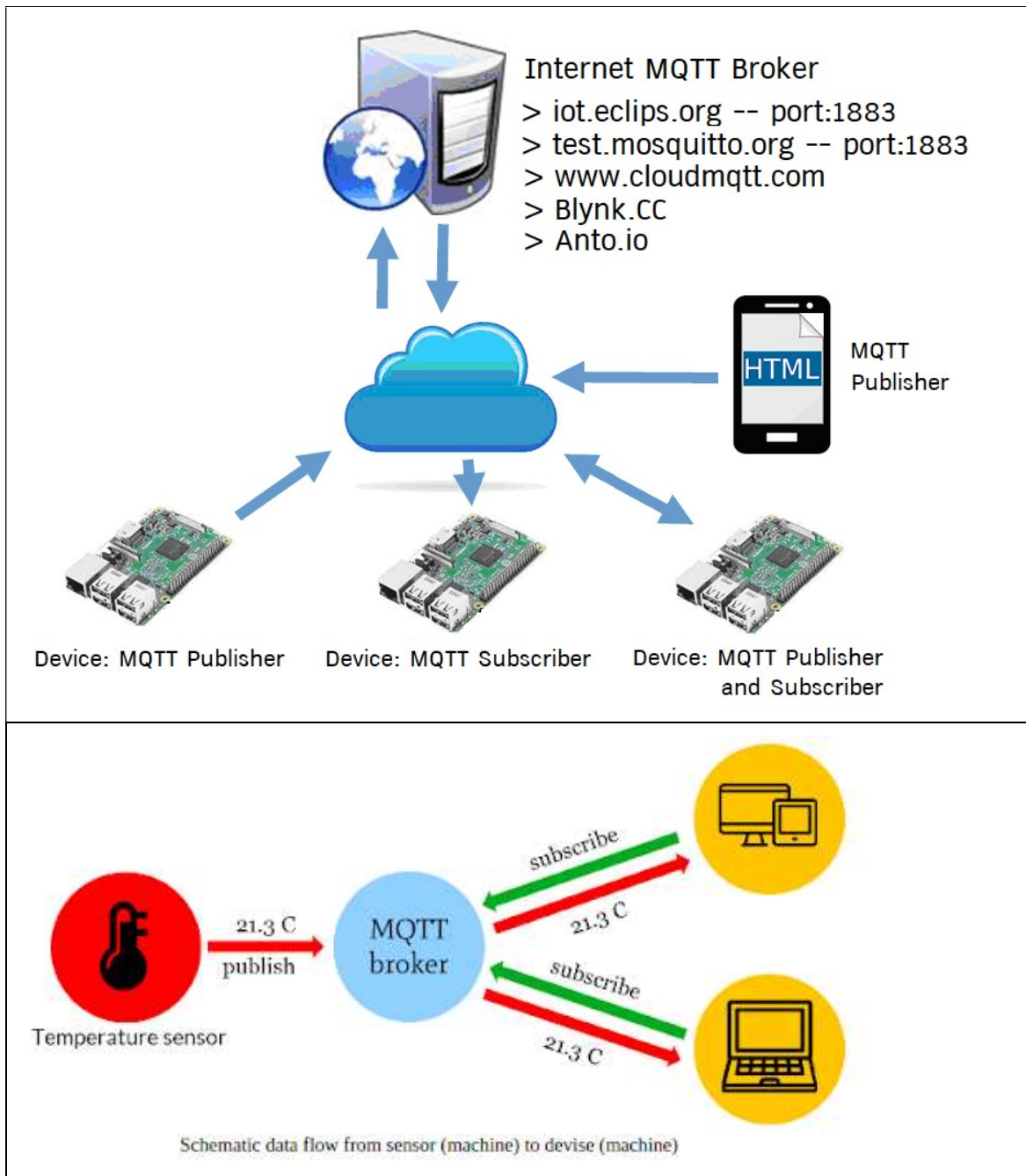
Q8. ให้นักศึกษาเขียนโปรแกรมเพื่อหาค่าสูงสุดของข้อมูลดังต่อไปนี้

- ค่าตำแหน่งหน่วยความจำเริ่มต้นของบล็อกข้อมูลในการค้นหาเก็บไว้ที่ Address 20H
- ค่าตำแหน่งหน่วยความจำสุดท้ายของบล็อกข้อมูลในการค้นหาเก็บไว้ที่ Address 21H
- เก็บตำแหน่งหน่วยความจำที่พบค่าสูงสุดที่ R6
- เก็บค่าสูงสุดที่พบ ใน R7

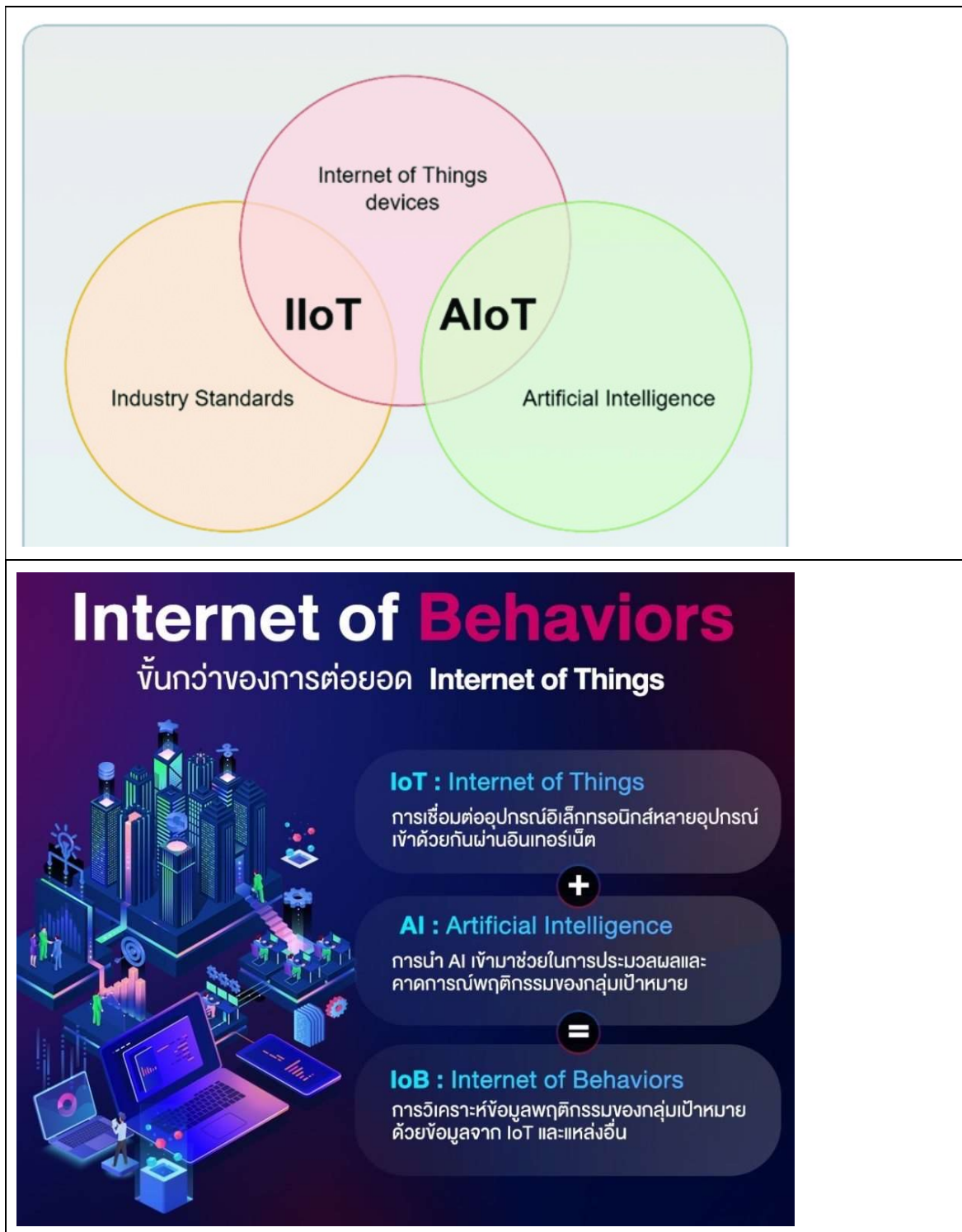
<pre> ORG 0000H MOV R7,#0H ; Start Max MOV R0,20H ; Start Addr INC 21H ; Stop+1 LOOP: MOV A,@R0 CLR C SUBB A,R7 JC _XCHK MOV A,R0 ; Address R6 MOV R6,A MOV A,@R0 ; Value R7 MOV R7,A _XCHK: INC R0 MOV A,R0 CJNE A,21H,LOOP SJMP \$ END </pre>	เริ่มจากค่าใน Addr.20H ถึงค่าใน Addr.21H หาก ตัวแปรX – ตัวแปรY แล้ว • ไม่มีการยืม $\rightarrow X < Y$ • มีการยืม $\rightarrow X \geq Y$
--	---

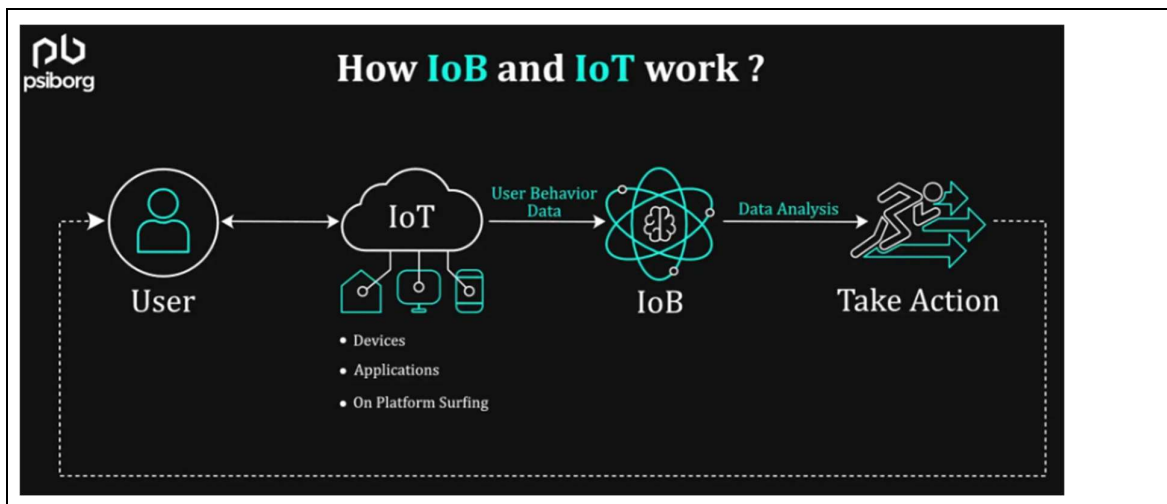
5/7: IoT, All Over IP, and Lab System

1.1 IoT Concept



1.2 IoT, IIoT, AIoT, IoB





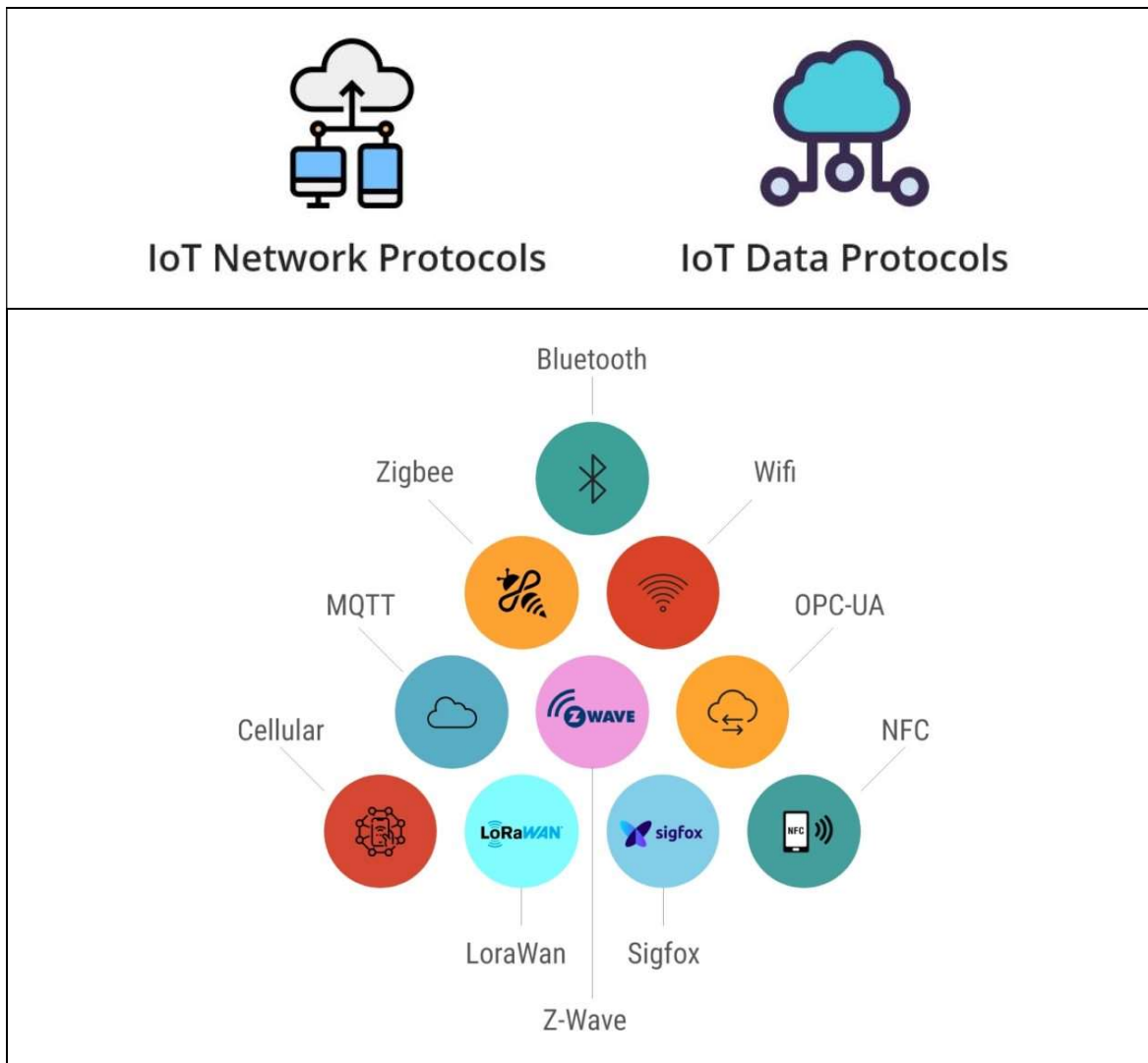
IoT (Internet of Things): หมายถึงเครือข่ายของอุปกรณ์ทางกายภาพ ("สิ่งของ") ที่ฝังด้วยเซ็นเซอร์ ซอฟต์แวร์ และเทคโนโลยีอื่นๆ เพื่อเชื่อมต่อและแลกเปลี่ยนข้อมูลกับอุปกรณ์และระบบอื่นๆ ผ่านอินเทอร์เน็ต เพื่อมุ่งเน้นการเชื่อมต่อและควบคุมอุปกรณ์ในชีวิตประจำวัน เพื่ออำนวยความสะดวกและความสะดวกสบาย เช่น บ้านอัจฉริยะ (smart home), อุปกรณ์สวมใส่ (wearable devices) เป็นต้น

IIoT (Industrial Internet of Things): เป็นส่วนย่อยของ IoT ที่มุ่งเน้นการใช้งานในภาคอุตสาหกรรม โดยใช้เซ็นเซอร์และอุปกรณ์เชื่อมต่อเพื่อเพิ่มประสิทธิภาพการผลิตและการดำเนินงาน เพื่อเพิ่มประสิทธิภาพการผลิต, ลดต้นทุน, ปรับปรุงความปลอดภัย, และเพิ่มความสามารถในการคาดการณ์ ตัวอย่างการใช้งาน เช่น โรงงานอัจฉริยะ (smart factories), การบำรุงรักษาเชิงคาดการณ์ (predictive maintenance), การจัดการซัพพลายเชน (supply chain management)

AIoT (Artificial Intelligence of Things): คือการรวมกันของปัญญาประดิษฐ์ (AI) และ IoT เพื่อให้อุปกรณ์ IoT สามารถวิเคราะห์และตัดสินใจได้ด้วยตัวเอง เพื่อเพิ่มความฉลาดและความสามารถในการคาดการณ์ของอุปกรณ์ IoT, ทำให้ระบบอัตโนมัติมีความซับซ้อนมากขึ้น, และปรับปรุงประสิทธิภาพโดยรวม ตัวอย่างการใช้งาน เช่น เมืองอัจฉริยะ (smart cities), การดูแลสุขภาพอัจฉริยะ (smart healthcare), ระบบขนส่งอัจฉริยะ (smart transportation)

IoB (Internet of Behaviors): เป็นแนวคิดที่ต่อยอดจาก IoT โดยมุ่งเน้นการวิเคราะห์ข้อมูลพฤติกรรมของผู้ใช้ที่ได้จากอุปกรณ์ IoT เพื่อเข้าใจและมีอิทธิพลต่อพฤติกรรมเหล่านั้น เพื่อเข้าใจพฤติกรรมผู้บริโภค, ปรับปรุงประสบการณ์ผู้ใช้, และสร้างแคมเปญการตลาดที่มีประสิทธิภาพ ตัวอย่างการใช้งาน เช่น การตลาดส่วนบุคคล (personalized marketing), การวิเคราะห์พฤติกรรมลูกค้า, การปรับปรุงการบริการ

1.3 Communication protocols for IoT



IoT Network Protocols

IoT (Internet of Things) network protocols play a crucial role in enabling seamless communication between medium to high-power devices within the IoT ecosystem. These protocols cater to the unique requirements of IoT devices, which often vary in power consumption, bandwidth, and communication range.

HTTP, LoRaWAN, Bluetooth, and ZigBee are some of the **popular IoT Network protocols**.

IoT Data Protocols

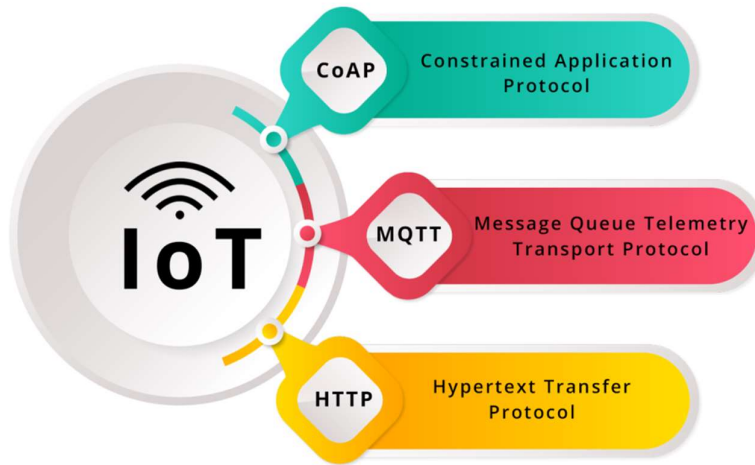
IoT data protocols are designed to connect low-power IoT devices. Without any internet connection, they are capable of providing end-to-end communication with the hardware.

Though the connectivity in IoT data protocols can be done via a wired or cellular network.

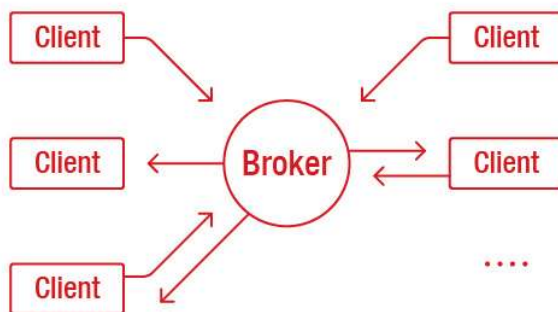
MQTT, CoAP, AMQP, and XMPP are some of the **popular IoT data protocols**.

1.4 Protocol in IoT: HTTP, MQTT, CoAP

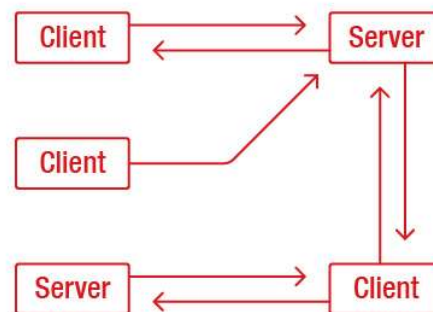
IoT Protocols



MQTT



CoAP



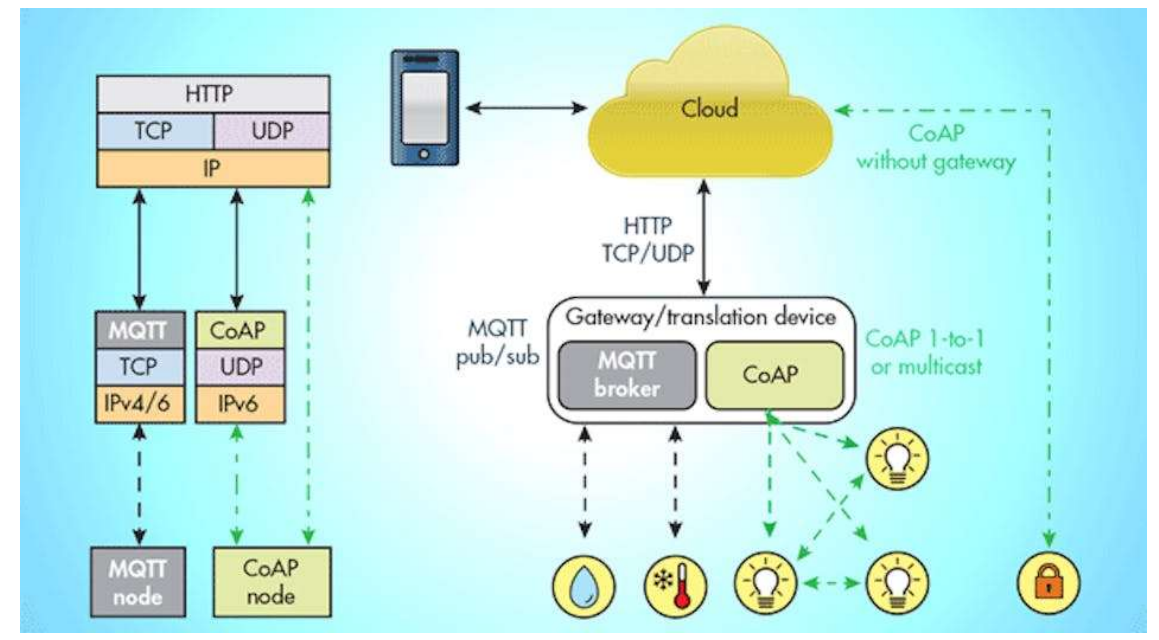
MQTT

MQTT (Message Queuing Telemetry Transport) เป็นโปรโตคอลที่ถูกออกแบบมาให้มีขนาดเล็กสำหรับการสื่อสารแบบ M2M (Machine to Machine) โดยถือกำเนิดจากวิศวกรจาก IBM และ Eurotech ในปี 1999 เพื่อนำไปใช้ในระบบ SCADA (Supervisory Control and Data Acquisition) สำหรับเชื่อมต่อท่อส่งน้ำมันบนเครือข่ายที่ไม่มีความเสถียรอย่างอินเทอร์เน็ตดาวเทียม ก่อนที่จะถูกปรับจัดกลายเป็น Open Standard ในปี 2014 โดย OASIS

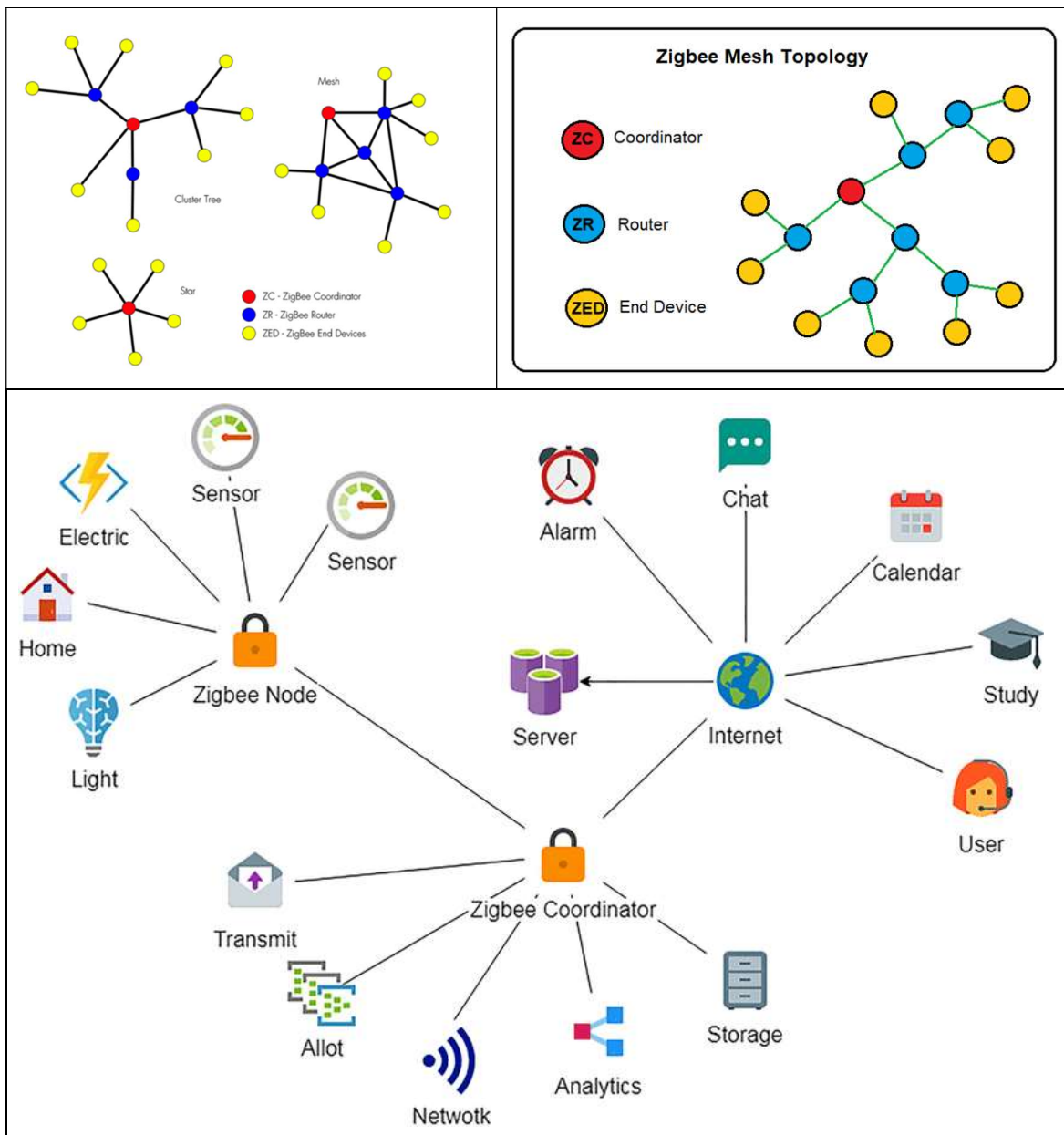
CoAP

CoAP (Constrained Application Protocol) เป็นมาตรฐานที่ถูกพัฒนาขึ้นมาใหม่โดย IETF ในปี 2014 โดยถูกออกแบบให้คล้ายกับ HTTP ซึ่งเป็น Document transfer protocol แต่มีขนาดเล็กกว่ามาก (มี header แบบคงที่ขนาด 4 byte) เพราะตัดส่วนที่ไม่จำเป็นทิ้งและรันบน UDP ซึ่งเป็น protocol ที่ไม่มีการสร้างการเชื่อมต่อระหว่างอุปกรณ์ปลายทาง จึงส่งข้อมูลได้เร็วมากแต่ไม่การันตีว่าข้อมูลจะถูกส่งไปยังปลายทางอย่างแน่นอนและถูกต้องตามลำดับ การส่งซ้ำและเรียงลำดับข้อมูลต้องไปทำบนระดับแอปพลิเคชัน

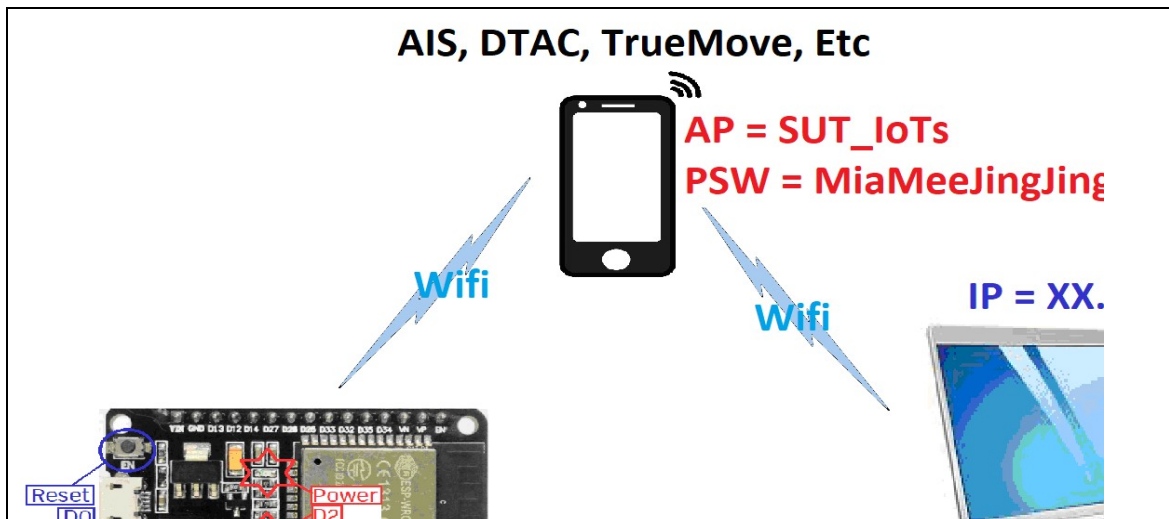
MQTT เป็นสถาปัตยกรรมแบบ Client/Server ซึ่งมี topology แบบ hub-and-spoke ค่ะ sensor ปลายทางจะทำหน้าที่เป็น client ซึ่งทำการสร้างเชื่อมต่อแบบ TCP ไปยัง Server ที่มีชื่อเรียกอีกชื่อว่า Broker ซึ่งมีหน้าที่เป็นเสมือนท่อส่งข้อมูลในการรับส่ง 'Message' ระหว่าง Client ที่เป็นได้ทั้ง Publisher และ Subscriberนั่นเอง Client – หมายถึง Publisher หรือ Subscriber ที่เชื่อมต่อแบบรวมศูนย์ไปยัง Broker ซึ่งสามารถเชื่อมต่อได้ทั้งแบบ persistent ที่ทำการสร้าง session ค้างไว้เปิดตลอดเวลาเพื่อติดต่อกับ Broker ซึ่งตรงกันข้ามกับ client ที่เชื่อมต่อแบบ transient ซึ่ง Broker ไม่สามารถติดตามสถานะได้ Broker – เป็น software ที่ทำหน้าที่รับข้อความทั้งหมดที่ได้จาก Publisher แล้วจึงส่งต่อไปให้ Subscriber ตามแต่ Topic ที่ client ได้ทำการ subscribe ไว้ Topic – เป็นเหมือน address หรือ endpoint บน Broker ที่ client ทำการเชื่อมต่อเพื่อรับส่งข้อความระหว่างกันนั่นเอง	CoAP เป็นสถาปัตยกรรมแบบ Client/Server โดย client จะทำการร้องขอทรัพยากรไปที่ server โดยตรง จากนั้น server จะทำการตอบกลับคำร้องพร้อมกับออปชัน 'Content-Type' เพื่อว่าบอก client ว่ากำลังจะได้รับข้อมูลในรูปแบบไหนกลับไป (เช่น JSON, XML, CBOR เป็นต้น) โดย client สามารถ GET, PUT, POST และ DELETE ทรัพยากรบน Server ด้วย URL และ query string คล้ายกับ REST API ที่เรารู้จักกันนั่นเอง ในการสถาปัตยกรรมแบบ CoAP ที่มีการแลกเปลี่ยนทรัพยากรกันโดยตรง Sensor Node ทำหน้าที่เป็นทั้ง Server และ Client ในเวลาเดียวกันเพราะต้องทำการตอบรับ packets ที่ถูกส่งมาหา
--	---



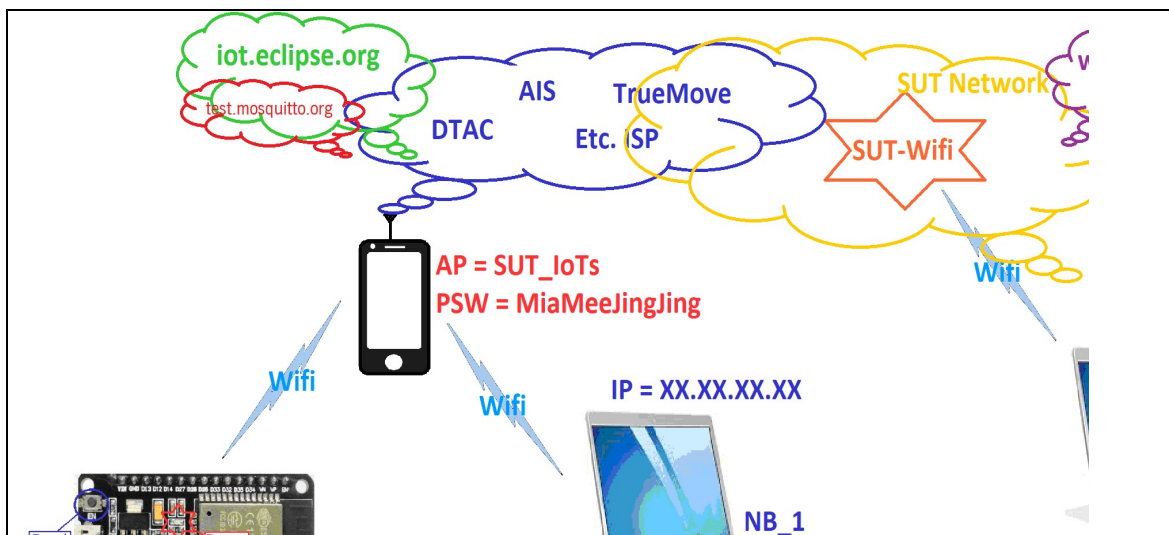
1.5 Zigbee



1.6 All Over IP



1.7 Lab System



6/7. MQTT Protocol

6.1 MQTT-Message Queue Telemetry Transport

โปรโตคอลที่ใช้สำหรับรับและส่งข้อมูลนั้นคือ MQTT ปัจจุบันถึง Version 3.1 ในที่นี้จะมาทำการทดลองส่งข้อมูลกันตัว Server จะมีอยู่ด้วยกันหลายค่ายครับสำหรับที่ลิสมาด้านล่างนี้ครับ

<u>Open Source MQTT Broker Server</u>	<u>Client</u>
• Mosquitto • RSMB • ActiveMQ • Apollo • Moquette • Mosca • RabbitMQ	• Paho • Xenqtt • mqtt.js • node_mqtt_client • Ascoltatori • Arduino MQTT Client

สำหรับ MQTT Broker Server ฟรีที่ใช้ทดสอบ ดังนี้

- test.mosquitto.org
- mqtt.eclipse.org
- broker.mqttdashboard.com

6.2 Free Public MQTT Brokers (Private & Public)

Posted by Maulin Tolia | Jan 30, 2019

<https://mntolia.com/10-free-public-private-mqtt-brokers-for-testing-prototyping/>

I have listed a few free public and private MQTT brokers that you can use for testing or prototyping.

Private MQTT Broker: In a private broker, only devices that you set can publish and subscribe to the topics on the broker. You should use this for production & prototyping.

Public MQTT Broker: In a public broker, any device can publish and subscribe to topics on the broker. There is no privacy. Public brokers should never be used in production. If you simply want to learn or play around with MQTT then use this.

If you want to install your own broker on your computer or virtual machine, see this:
Installing Mosquitto Broker on Ubuntu/Windows/Debian

The information in the tables were last updated on April 7th 2020.

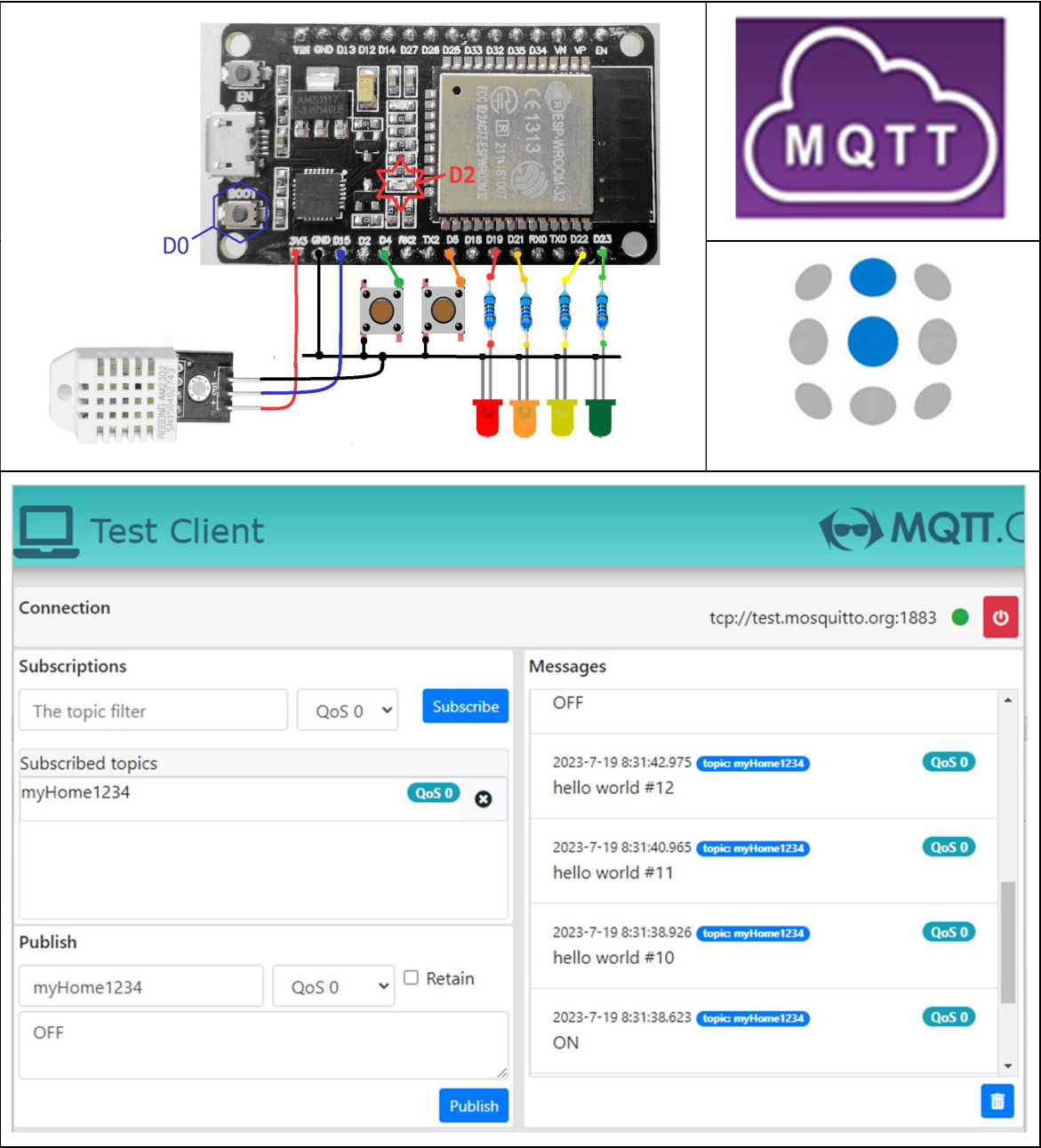
Public MQTT Brokers

Name	Broker Address	TCP Port	TLS Port	WebSocket Port	Message Retention	Persistent Session
Eclipse	mqtt.eclipse.org	1883	N/A	80, 443	YES	YES
Mosquitto	test.mosquitto.org	1883	8883, 8884	80	YES	YES
HiveMQ	broker.hivemq.com	1883	N/A	8000	YES	YES
Flespi	mqtt.flespi.io	1883	8883	80, 443	YES	YES
Dioty	mqtt.dioty.co	1883	8883	8080, 8880	YES	YES
Fluux	mqtt.fluux.io	1883	8883	N/A	N/A	N/A
EMQX	broker.emqx.io	1883	8883	8083	YES	YES
Eclipse	mqtt.eclipse.org	1883	N/A	80, 443	YES	YES

Private MQTT Brokers

Name	Broker Address	TCP Port	TLS Port	WebSocket Port	Message Retention	Persistent Session
Azure	NO	8883	443	NO	Limited	0, 1
AWS	NO	8883	443	NO	Limited	0, 1
CloudMQTT	Custom Port	Custom Port	Custom Port	NOT SURE	YES	0, 1, 2

6.3 การทดสอบ



The image shows a hardware setup for testing MQTT communication. An ESP8266 NodeMCU board is connected to a DHT22 sensor. The board's pins are labeled D0, D1, D2, D3, D4, D5, D6, D7, D8, D9, D10, D11, D12, D13, D14, D15, D16, D17, D18, D19, D20, D21, D22, D23. A red star is drawn on the board near the D2 pin. To the right of the board are the MQTT logo (a purple cloud with 'MQTT' text) and the Blyn IoT logo (a grid of blue and grey dots). Below these is a screenshot of the MQTT Test Client web interface. The interface shows a connection to tcp://test.mosquitto.org:1883. Under 'Subscriptions', the topic 'myHome1234' is listed with QoS 0. Under 'Publish', the topic 'myHome1234' is entered with QoS 0 and the message 'OFF'. The 'Messages' list shows four messages: 'hello world #12', 'hello world #11', 'hello world #10', and 'ON', all on the 'myHome1234' topic with QoS 0.

- <https://testclient-cloud.mqtt.cool/> → Web Base Client
- <https://mqtt-explorer.com/> → Application for Download Install and Run
- Note: ระวางเรื่องการปิดพอร์ต 1883 จากผู้ดูแลเครือข่ายอินเทอร์เน็ต

7/7. ESP32 – BLE Remote

1. Read This: <https://ai.thestempedia.com/docs/dabble-app/getting-started-with-dabble/>
2. Install Mobile Application
 - Android: <https://play.google.com/store/apps/details?id=io.dabbleapp>
 - IOS: <https://apps.apple.com/us/app/dabble-bluetooth-controller/id1472734455>
3. Install **ESP32 Library** → **DabbleESP32** by Mimansa Maheshwari Version 1.5.1

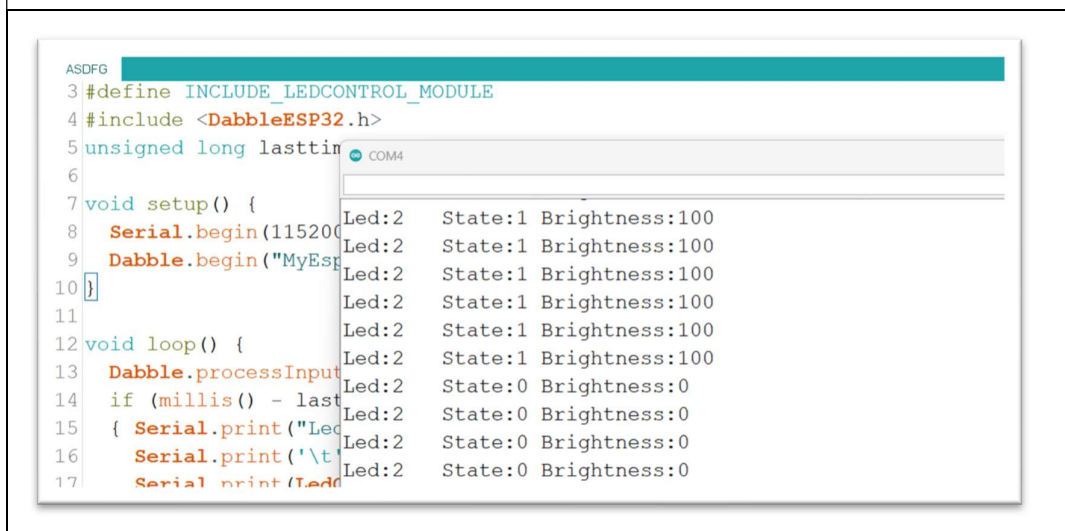


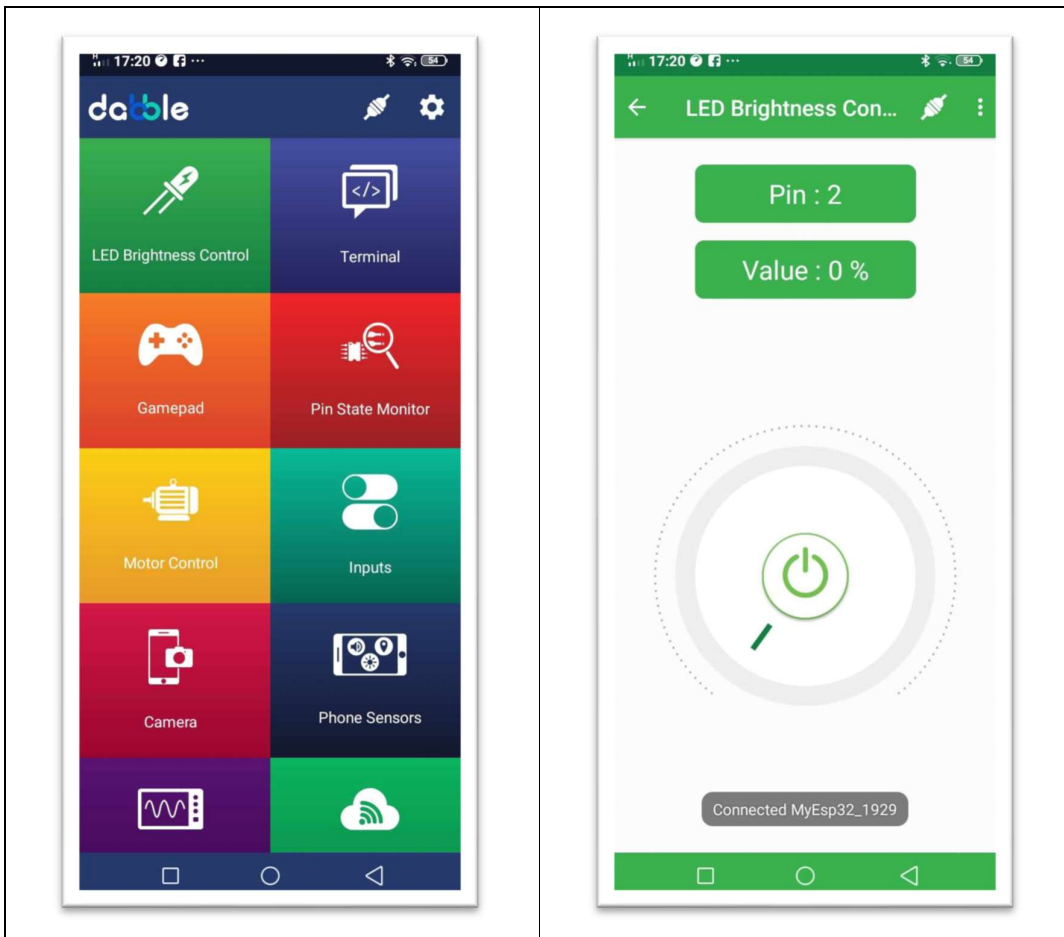
4. Test LED On/Off: File → Example → DabbleESP32 → 01LEDBrightControl

```
#define CUSTOM_SETTINGS
#define INCLUDE_LEDCONTROL_MODULE
#include <DabbleESP32.h>
unsigned long lasttime;

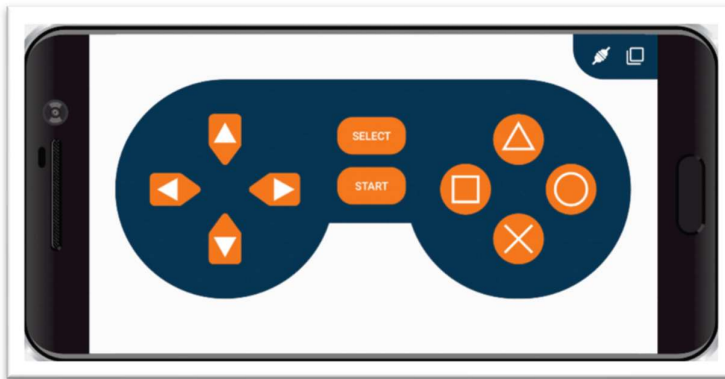
void setup() {
  Serial.begin(115200);
  Dabble.begin("MyEsp32_1929");
}

void loop() {
  Dabble.processInput();
  if (millis() - lasttime >= 250)
  { Serial.print("Led:"); Serial.print(LedControl.getpinNumber());
    Serial.print('\t'); Serial.print("State:"); // 0=Off. 1=On.
    Serial.print(LedControl.getpinState());
    Serial.print('\t'); Serial.print("Brightness:");
    Serial.println(LedControl.readBrightness());
    lasttime = millis();
  }
}
```





5. ทดสอบการใช้งาน Dabble Joystick



```
#define CUSTOM_SETTINGS
#define INCLUDE_GAMEPAD_MODULE
#include < DabbleESP32.h>
#define BLEDeviceName "MyEsp32_1929"
unsigned long lasttime;

void setup() {
  Serial.begin(115200);
  Dabble.begin(BLEDeviceName);
  Serial.println(BLEDeviceName);
}

void loop() {
  Dabble.processInput();
  if (millis() - lasttime >= 250) {
    lasttime = millis();
    Serial.print("\nKeyPressed: ");
    int Ang = GamePad.getAngle();
    int Rad = GamePad.getRadius();
    float XAis = GamePad.getXaxisData();
    float YAis = GamePad.getYaxisData();
    Serial.print("\tAngle: ");    Serial.print(Ang);
    Serial.print("\tRadius: ");   Serial.print(Rad);
    Serial.print("\tx_axis: ");   Serial.print(XAis);
    Serial.print("\ty_axis: ");   Serial.print(YAis);
    if (GamePad.isUpPressed())    Serial.print("\tUp");
    if (GamePad.isDownPressed())  Serial.print("\tDown");
    if (GamePad.isLeftPressed())  Serial.print("\tLeft");
    if (GamePad.isRightPressed()) Serial.print("\tRight");
    if (GamePad.isSquarePressed()) Serial.print("\tSquare");
    if (GamePad.isCirclePressed()) Serial.print("\tCircle");
    if (GamePad.isCrossPressed()) Serial.print("\tCross");
    if (GamePad.isTrianglePressed()) Serial.print("\tTriangle");
    if (GamePad.isStartPressed()) Serial.print("\tStart");
    if (GamePad.isSelectPressed()) Serial.print("\tSelect");
  }
}
```

6. ปรับปรุงโปรแกรมเพื่อควบคุม 8 LED แบบกดติดกดดับ ด้วย Dabble App



```
#define CUSTOM_SETTINGS
#define INCLUDE_GAMEPAD_MODULE
#include <DabbleESP32.h>
#define BLEDeviceName "MyEsp32_1929"
unsigned long lasttime;
#define pinTest1 15
#define pinTest2 2
int stsPinTest1 = 0;
int stsPinTest2 = 0;
bool chkPress = true;

void setup() {
  Serial.begin(115200);
  Dabble.begin(BLEDeviceName);
  Serial.println(BLEDeviceName);
  pinMode(pinTest1, OUTPUT);
  pinMode(pinTest2, OUTPUT);
}

void loop() {
  Dabble.processInput();
  digitalWrite(pinTest1, stsPinTest1);
  digitalWrite(pinTest2, stsPinTest2);

  if (millis() - lasttime > 250) {
    lasttime = millis();

    if (GamePad.isUpPressed()) {
      Serial.println("Key Up");
      stsPinTest1 = 1 - stsPinTest1;
      chkPress = true;
      while (chkPress) {
        delay(50);
        Dabble.processInput();
        chkPress = GamePad.isUpPressed();
      }
    }

    if (GamePad.isTrianglePressed()) {
      Serial.println("Key Triangle");
      stsPinTest2 = 1 - stsPinTest2;
      chkPress = true;
      while (chkPress) {
        delay(50);
        Dabble.processInput();
        chkPress = GamePad.isTrianglePressed();
      }
    }
  }
}
```

7. แนวการต่อวงจรเพื่อทดสอบ

